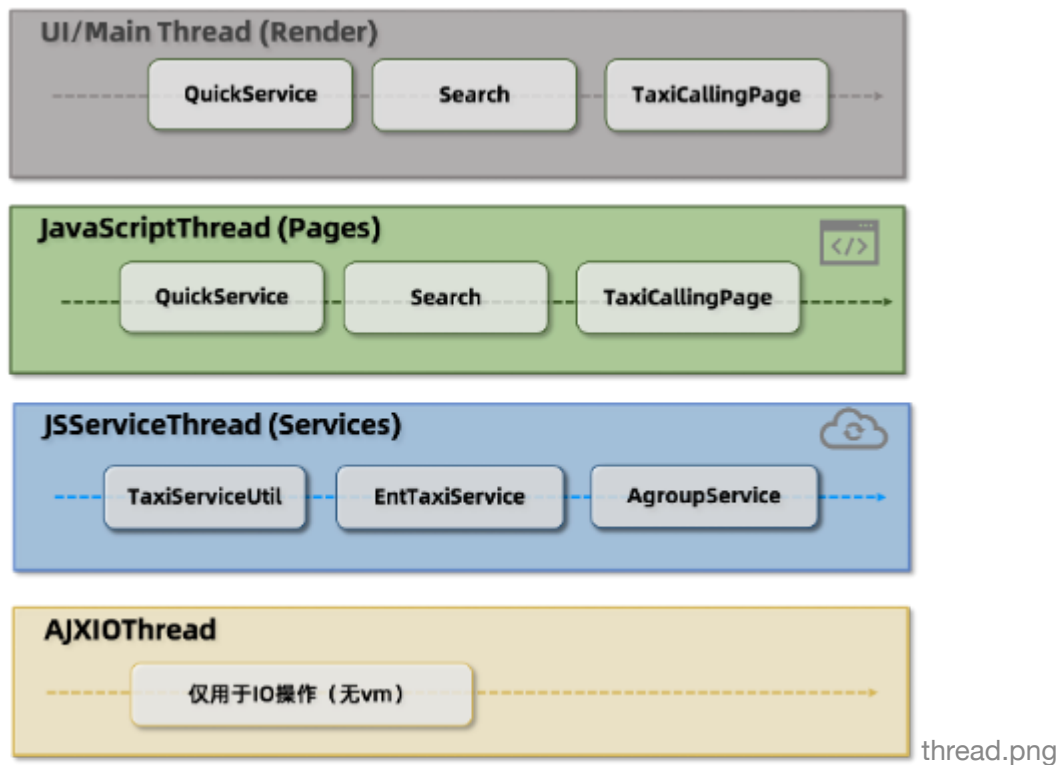
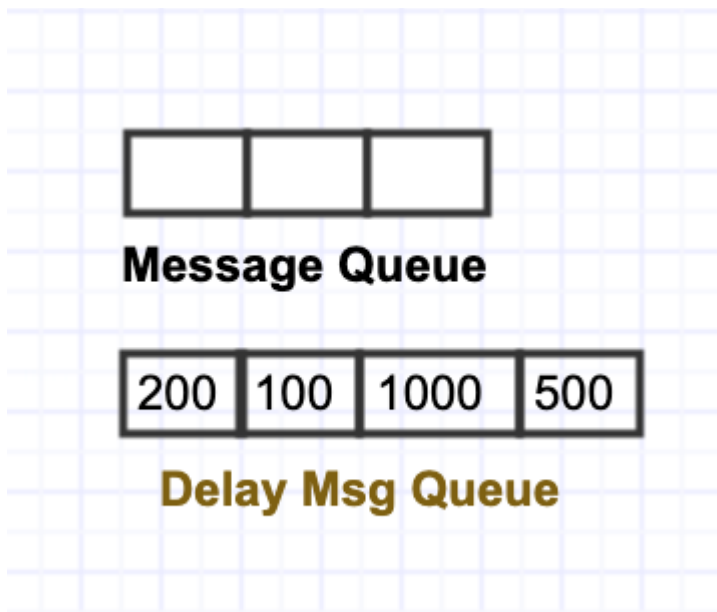


引擎基本介绍

线程模型

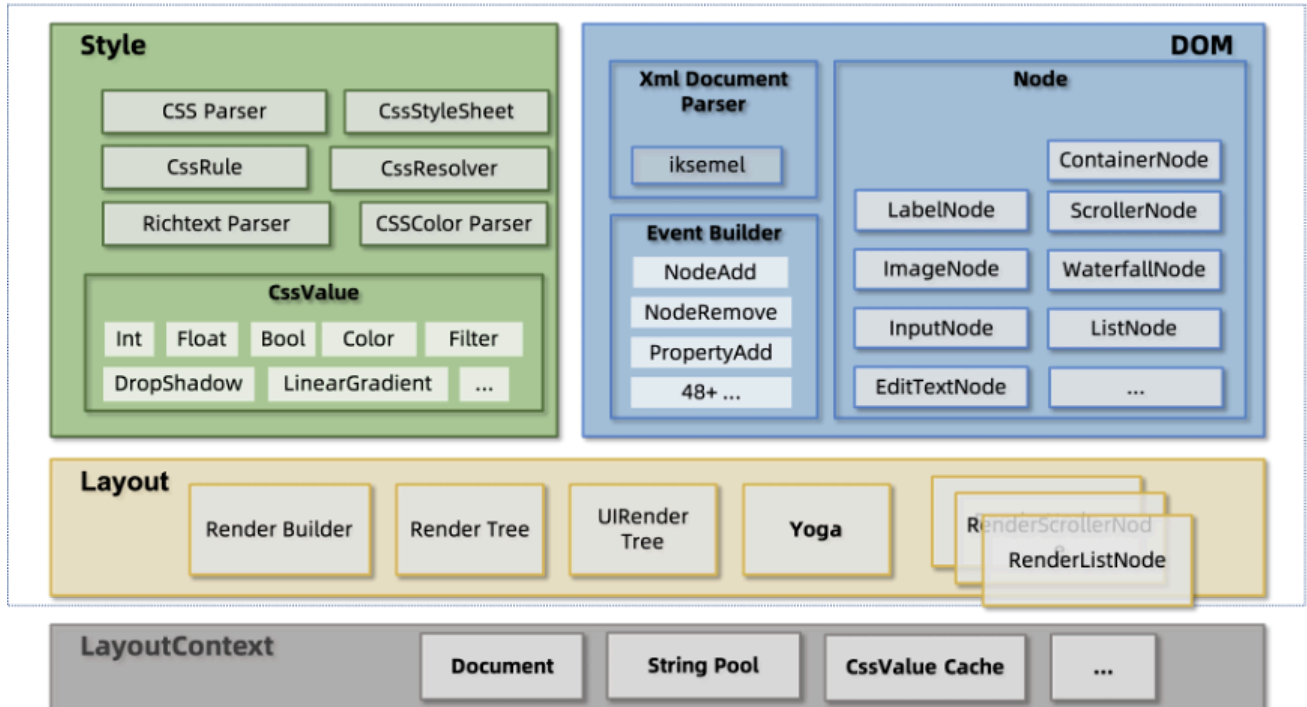


- UI线程 (main/Main)
 - UI线程也被称为主线程，是由Android/iOS平台创建的，所有View的更新都需要在UI线程完成
- JS线程 (JavaScriptThread)
 - JavaScript代码和环境需要独立运行在JS线程中，该线程是串行的，仅一个线程由引擎创建，即JS线程（浏览器中也是如此）
- Service线程 (JsServiceThread)
 - 含虚拟机实例，所有的AJX Service均共享此线程。
- IO线程 (AJXIOThread)
 - 执行IO操作的独立线程，IO线程不存在JS虚拟机实例，因此不能执行JS代码
- 以上线程除UI线程外，均是由AJX引擎在初始化时创建的。其中JS线程和服务线程都包含虚拟机实例，可以用来并发执行JS脚本。



- 所有线程都使用了消息机制，其中UI线程中的消息管理是由双端平台完成的。其他线程中的消息机制是在AJX引擎中实现的。消息分为“即时消息”和“延时消息”两种，分别在不同的消息队列中进行管理。ajax.setTimeout便是典型的延时消息的使用场景。不同线程之间的通信也使用了消息机制。比方异步的NativeModule调用便是在JS线程中向UI线程的消息队列投递了一个消息，这个消息本质上是一个可执行的函数。

布局计算



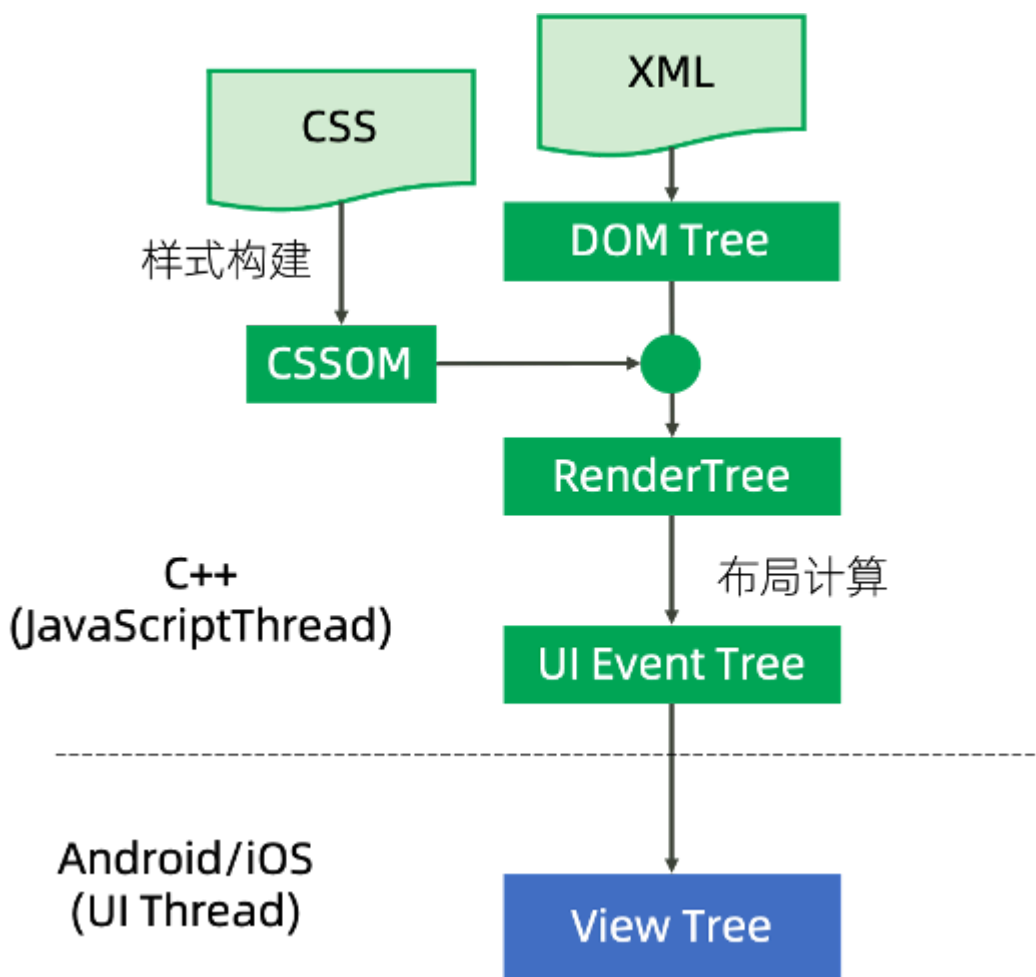
layout_engine.png

AJX容器引擎中的布局计算模块是由C++实现的，各模块简介：

- **Style：** 样式解析与匹配模块。

- DOM: 文档对象模型，使用了开源工程iksemel进行xml的解析，用于生成DOM树。定义了Node作为节点基类，ContainerNode继承Node用于表示容器节点。Event Builder用于最终生成UI事件给适配层渲染。
- Layout: 通过DOM树与CSSOM结合生成RenderTree进行布局的计算。list中使用了DOM“模板”做为关键的性能提升技术，在相同DOM结构时计算布局和渲染时可以做到模版复用。
- LayoutContext: 即布局上下文的概念。LayoutContext中的Document实例封装了根节点和样式表，StringPool属于性能优化点，提供了字符串缓存池，避免了大量的string对象的频繁创建和销毁，在LayoutContext销毁时统一清空缓存。

布局计算流程示意图



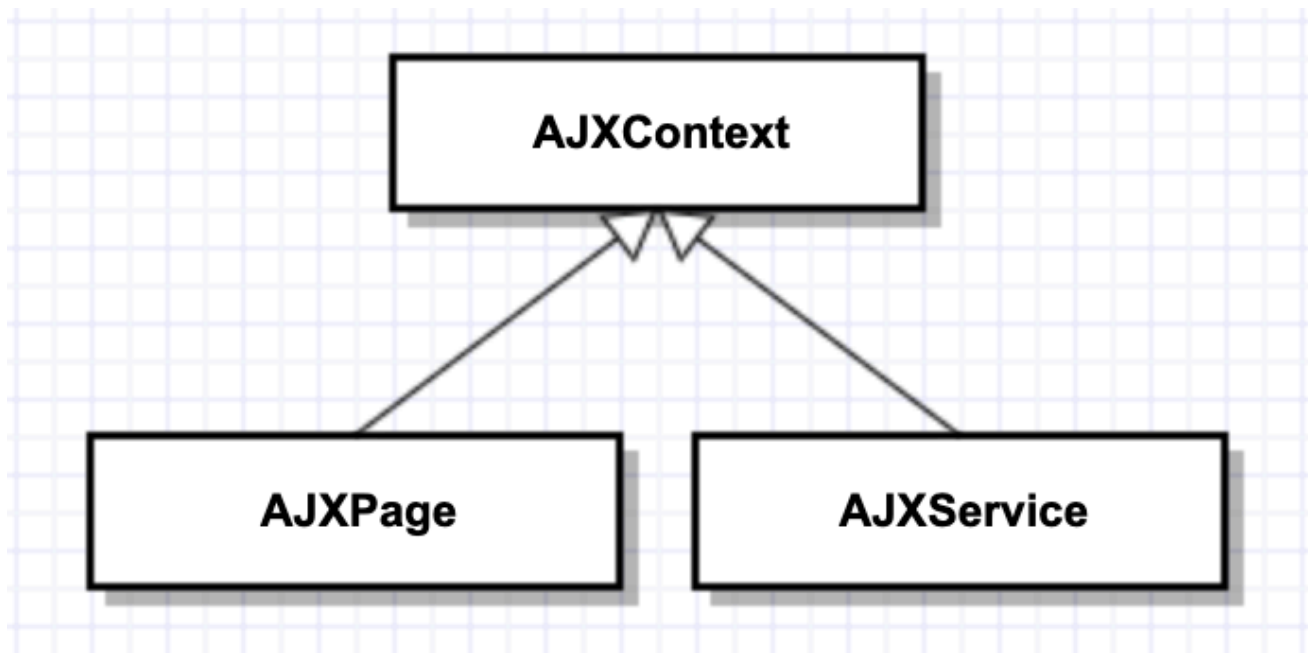
JS运行环境

JavaScript引擎

- JavaScript 引擎是互联网世界的基础设施。AJX引擎在iOS端使用了系统预置的[JavaScriptCore](#)引擎，Android在调试包中使用了[v8](#)引擎，非调试包也使用了[JavaScriptCore](#)引擎。在Android平台上AJX引擎有能力任意切换JavaScriptCore和v8引擎。
- 当前，所有平台以及所有包类型全部使用了集团基于[QuickJs](#)的[gking](#)引擎。

AJXContext

- 在逻辑上AJX引擎封装了AJXPage和AJXService两种上下文，分别对应页面和后台服务。



ajxcontext.png

base.js

- base.js 内嵌到了AJX引擎中，提供按需加载 module，增强运行环境 API 功能，这个文件是预置在ajx引擎里的；调试时是可以看到这个文件的。